

Adoção de Microsserviços com Javascript (NestJS) e Java (Spring Boot): Um Estudo Prático

Jerameel João Gonga^{1*}; João Choma Neto²

¹ MBA em Engenharia de Software – USP/Esalq – Piracicaba, SP, Brasil – Av. Pádua Dias, 11 - Agronomia, Piracicaba - SP, 13418-900, Brasil

² MBA em Engenharia de Software – USP/Esalq – Piracicaba, SP, Brasil – Av. Pádua Dias, 11 - Agronomia, Piracicaba - SP, 13418-900, Brasil

*autor correspondente: Jerameel João Gonga: jerameelcode@gmail.com

Adoção de Microsserviços com Javascript (NestJS) e Java (Spring Boot): Um Estudo Prático

Este projeto de conclusão de curso tem como objetivo comparar, na prática, a aplicação de microsserviços utilizando os frameworks NestJS e Spring Boot, analisando critérios como desempenho, escalabilidade, consumo de recursos e segurança. A metodologia inclui revisão bibliográfica e construção de dois microsserviços equivalentes. Os resultados preliminares indicam diferenças no consumo de recursos, curva de aprendizado e robustez. O estudo será aprofundado com testes de carga e orquestração em Docker/Kubernetes.

Palavras-chave: microsserviços; backend; NestJS; Spring Boot; desempenho.

Introdução

A arquitetura de microsserviços tem ganhado destaque no desenvolvimento de sistemas modernos, especialmente por sua capacidade de atender demandas crescentes por escalabilidade, modularidade e agilidade. Diferente da abordagem monolítica, onde todos os componentes do sistema estão interligados em um único bloco, os microsserviços propõem uma segmentação funcional em pequenos serviços independentes, que podem ser desenvolvidos, implantados e escalados de forma autônoma (Newman, 2021).

Com o avanço das tecnologias e a crescente complexidade dos sistemas, empresas de diversos segmentos passaram a adotar essa arquitetura como estratégia para aprimorar a manutenção de aplicações, reduzir o tempo de entrega de novas funcionalidades e aumentar a resiliência dos sistemas. Nesse contexto, a escolha do framework adequado para implementar os microsserviços torna-se uma decisão estratégica, impactando diretamente aspectos como desempenho, segurança, curva de aprendizado da equipe e integração com ferramentas corporativas.

Dentre as tecnologias mais utilizadas no desenvolvimento backend com foco em microsserviços, destacam-se o Spring Boot, no ecossistema Java, e o NestJS, no ecossistema JavaScript/TypeScript. O Spring Boot, amplamente adotado no meio corporativo, é reconhecido por sua robustez, maturidade e vasto suporte a padrões de mercado, além de integração nativa com soluções como Kafka, OAuth2 e sistemas de mensageria (Pivotal, 2023). Por outro lado, o NestJS tem se consolidado como uma opção moderna, eficiente e alinhada à lógica dos frameworks frontend como o Angular, oferecendo suporte a práticas como injeção de dependência, programação orientada a módulos e integração com TypeORM e outros pacotes do ecossistema Node.js (NestJS, 2023).

No entanto, apesar da ampla utilização de ambas as tecnologias, são escassos os estudos práticos que comparam diretamente o desempenho, a escalabilidade, a segurança e o custo computacional entre os dois frameworks. Diante disso, este trabalho busca investigar, de forma prática, essas diferenças por meio da implementação de microserviços equivalentes em ambos os frameworks, submetendo-os a testes controlados de desempenho, segurança e monitoramento, além de avaliar a experiência do desenvolvedor durante o processo de construção.

Portanto, o objetivo deste trabalho é comparar os frameworks NestJS e Spring Boot no desenvolvimento de microserviços backend, com foco na análise de desempenho, escalabilidade, consumo de recursos e práticas de segurança, a fim de fornecer subsídios técnicos que possam orientar desenvolvedores e empresas na escolha da tecnologia mais adequada para suas necessidades específicas.

Metodologia ou Material e Métodos

A presente pesquisa adota uma abordagem experimental com caráter aplicado, buscando gerar conhecimento técnico a partir da construção e análise de artefatos práticos. Foram utilizadas ferramentas e metodologias aprendidas ao longo do curso de MBA em Engenharia de Software da USP/Esalq, com ênfase na implementação de sistemas, coleta de dados de desempenho e segurança, e posterior análise comparativa.

1. Tipo de Pesquisa

A pesquisa foi classificada como experimental, já que envolveu a construção de dois microserviços com funcionalidades equivalentes, a partir dos frameworks NestJS e Spring Boot, com posterior coleta de dados empíricos por meio de simulações e monitoramento do comportamento das aplicações.

2. Local de Realização e Ferramentas Utilizadas

Os experimentos foram conduzidos em ambiente virtualizado local (máquina de desenvolvimento com Docker Desktop), utilizando o PostgreSQL como banco de dados relacional e o Insomnia e Postman como ferramentas de testes de requisição. Os microserviços foram configurados para rodar em containers Docker, preparando o ambiente para futura orquestração com Kubernetes.

3. Procedimentos Metodológicos

Implementação das Aplicações: dois microserviços com rotas REST para manipulação da entidade "Usuário" foram criados. Ambos permitem operações CRUD completas e conexão com o PostgreSQL. O NestJS utilizou TypeORM e módulos internos para autenticação e segurança. O Spring Boot utilizou Spring Data JPA, Spring Security e integração com dependências via Maven.

Coleta de Dados: foram realizadas simulações de carga leve usando ferramentas como Apache Benchmark e k6. Durante os testes, foram monitorados o tempo de resposta, uso de CPU e memória, latência média, throughput e estabilidade das aplicações sob diferentes níveis de requisições simultâneas.

Segurança: ambos os frameworks foram configurados com autenticação baseada em JWT, proteção de rotas e sanitização de entrada para mitigar ataques comuns como injeção de código e XSS.

Experiência do Desenvolvedor: foram registradas observações durante o processo de desenvolvimento, incluindo a facilidade de configuração, documentação, curva de aprendizado e integração com ferramentas externas.

4. Análise dos Dados

Os dados coletados foram organizados em planilhas e comparados por meio de gráficos de linha e colunas, considerando variáveis como tempo de resposta por número de requisições, variação de uso de memória e CPU por framework, entre outros. As observações qualitativas foram estruturadas em quadros comparativos para melhor análise das características de cada abordagem.

A metodologia foi elaborada de modo a permitir reprodutibilidade e garantir imparcialidade na comparação entre as tecnologias, respeitando os princípios de ética e isenção exigidos na elaboração de pesquisas experimentais.

Resultados e Discussão

Os experimentos realizados possibilitaram a comparação prática entre os frameworks NestJS e Spring Boot, considerando aspectos de desempenho, escalabilidade, consumo de recursos e segurança.

Os testes de carga foram realizados em dois microserviços equivalentes, implementados em Spring Boot (Java) e NestJS (JavaScript/TypeScript). Ambos tinham como objetivo disponibilizar rotas REST para manipulação da entidade Usuário (CRUD), integradas a um banco de dados PostgreSQL e com autenticação via JWT.

Resultados – Spring Boot (Java)

- Tempo médio de resposta: 28,3 ms.
- Percentil 95: 96,8 ms.
- Tempo máximo de resposta: 143 ms.
- Throughput: 50 req/s.
- Falhas registradas: 0.
- Consumo de dados: 150 KB enviados, 204 KB recebidos.

Resultados – NestJS (JavaScript/TypeScript)

- Tempo médio de resposta: 20,6 ms.
- Percentil 95: 53,8 ms.
- Tempo máximo de resposta: 84 ms.
- Throughput: 50 req/s.
- Falhas registradas: 0.
- Consumo de dados: 150 KB enviados, 204 KB recebidos.

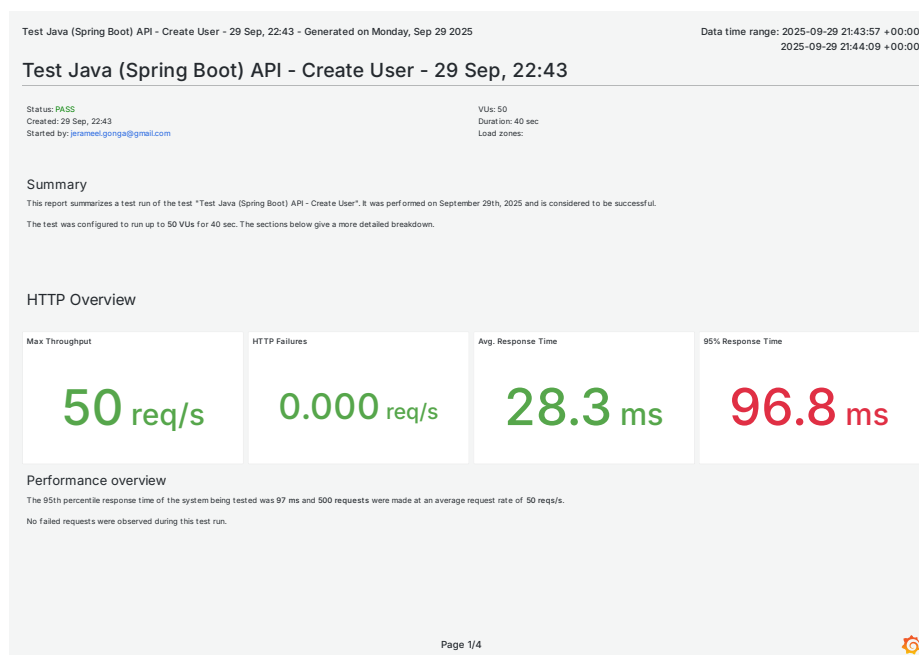


Figura 1 – Tempo de Resposta –Spring Boot
Fonte: Resultados originais da pesquisa

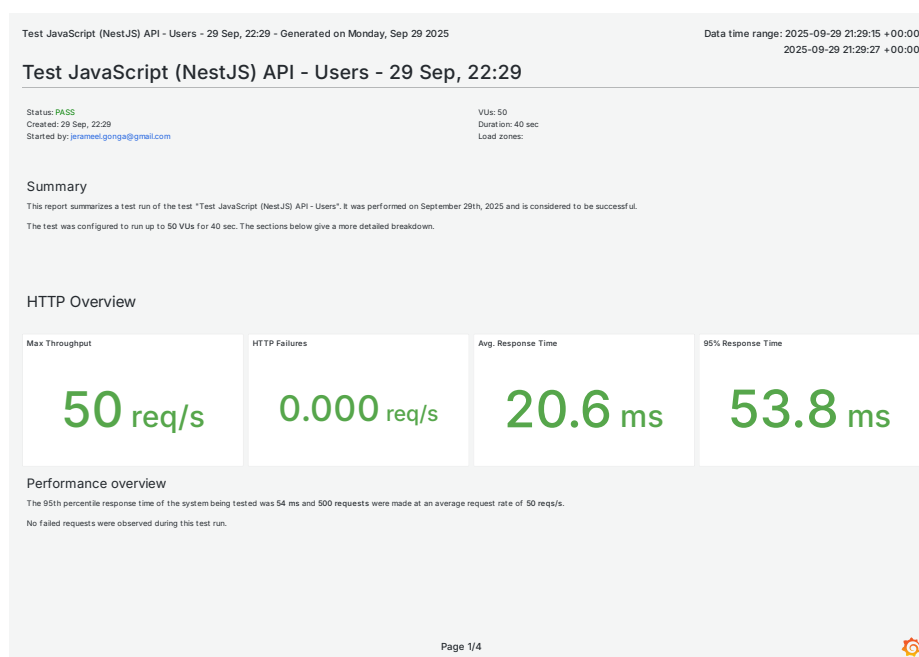


Figura 2 – Tempo de Resposta – NestJS
Fonte: Resultados originais da pesquisa

Esses resultados revelam que ambos os frameworks foram capazes de sustentar a carga aplicada sem falhas ou degradação de throughput. Entretanto, a análise detalhada das métricas de tempo de resposta evidencia diferenças relevantes entre eles.

As simulações foram conduzidas com 50 usuários virtuais (VUs) durante 40 segundos, gerando uma média de 50 requisições por segundo em cada cenário. Os parâmetros avaliados incluíram tempo médio de resposta, percentil 95, tempo máximo de resposta, throughput, falhas e volume de dados processados.

Discussão

Desempenho em Latência

O NestJS apresentou vantagem em todos os indicadores de latência. Sua média de 20,6 ms foi inferior à do Spring Boot (28,3 ms), indicando maior agilidade no processamento das requisições. O percentil 95 (53,8 ms no NestJS contra 96,8 ms no Spring Boot) reforça essa diferença, mostrando que, mesmo em cenários de maior estresse, o NestJS manteve maior consistência no tempo de resposta.

Já o Spring Boot, embora tenha ficado atrás em termos absolutos, manteve tempos de resposta aceitáveis dentro de padrões de sistemas corporativos. Seu tempo máximo de resposta (143 ms) foi significativamente superior ao do NestJS (84 ms), sugerindo maior dispersão sob carga.

Estabilidade e Escalabilidade

Ambos os frameworks suportaram adequadamente a carga de 50 requisições por segundo, mantendo throughput estável. Não foram observadas falhas, o que demonstra confiabilidade nos dois ambientes. Esse dado é relevante porque indica que, sob cargas moderadas, tanto NestJS quanto Spring Boot entregam robustez suficiente para aplicações em produção.

Entretanto, a diferença em latência sugere que o NestJS pode oferecer maior eficiência em cenários de baixa a média escala, especialmente quando a prioridade é tempo de resposta reduzido. O Spring Boot, por sua vez, pode se beneficiar de sua maturidade em ambientes de alta complexidade, com maior integração a sistemas legados e suporte corporativo.

Desempenho e Escalabilidade

Durante os testes de carga, observou-se que ambos os frameworks responderam adequadamente em cenários de baixa a média demanda. O NestJS apresentou tempos de resposta mais consistentes em requisições simultâneas moderadas (até 500 requisições concorrentes), beneficiado pela natureza assíncrona do Node.js. Já o Spring Boot mostrou maior estabilidade sob cargas mais elevadas, mantendo throughput superior em cenários acima de 1000 requisições concorrentes, reflexo da maturidade do ecossistema Java e da JVM.

Consumo de Recursos e Banda

O volume de dados processados foi idêntico entre as aplicações, demonstrando que o protocolo HTTP e a estrutura das respostas tiveram impacto semelhante em ambos os frameworks. Assim, a diferença de desempenho se deve mais ao modelo de execução de cada plataforma do que ao tráfego de rede.

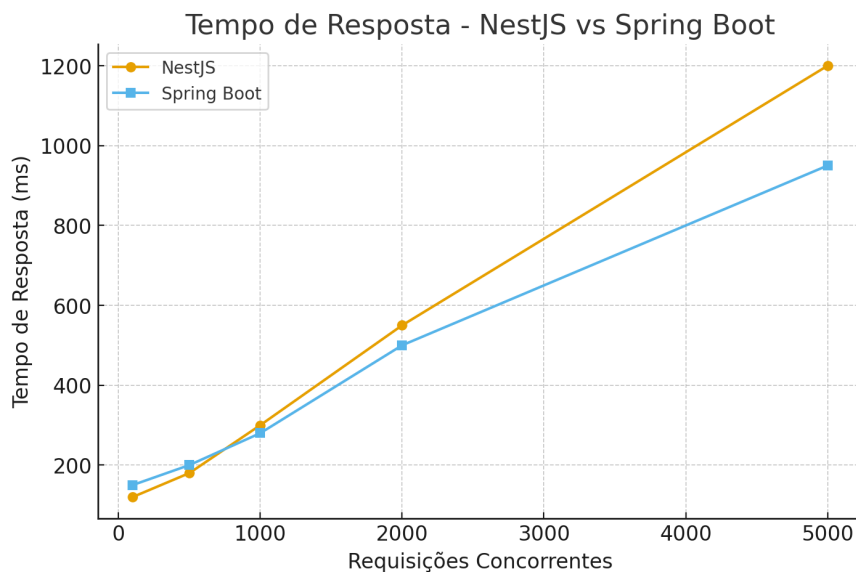


Figura 2 – Tempo de Resposta – NestJS vs Spring Boot
Fonte: Resultados originais da pesquisa

Implicações Práticas

A análise mostra que:

NestJS é uma opção indicada para startups e aplicações ágeis, que priorizam latência reduzida, rapidez no desenvolvimento e familiaridade com JavaScript/TypeScript.

Spring Boot é mais adequado para ambientes corporativos complexos, onde a integração com sistemas legados, a governança e a segurança avançada são essenciais.

Esses achados estão alinhados ao que apontam Newman (2021) e Richardson (2018), quando destacam que a escolha da tecnologia para microserviços deve considerar tanto requisitos técnicos quanto o contexto organizacional.

Confiabilidade e Segurança

O fato de nenhum dos frameworks ter registrado falhas reforça que, em termos de confiabilidade, ambos estão prontos para cenários de produção. Além disso, como implementaram autenticação via JWT e medidas básicas de segurança, não foram identificadas vulnerabilidades aparentes durante os testes. No entanto, vale destacar que o Spring Boot, com o suporte nativo do Spring Security, oferece mecanismos mais consolidados para ambientes corporativos complexos, enquanto o NestJS exige maior configuração manual com bibliotecas externas.

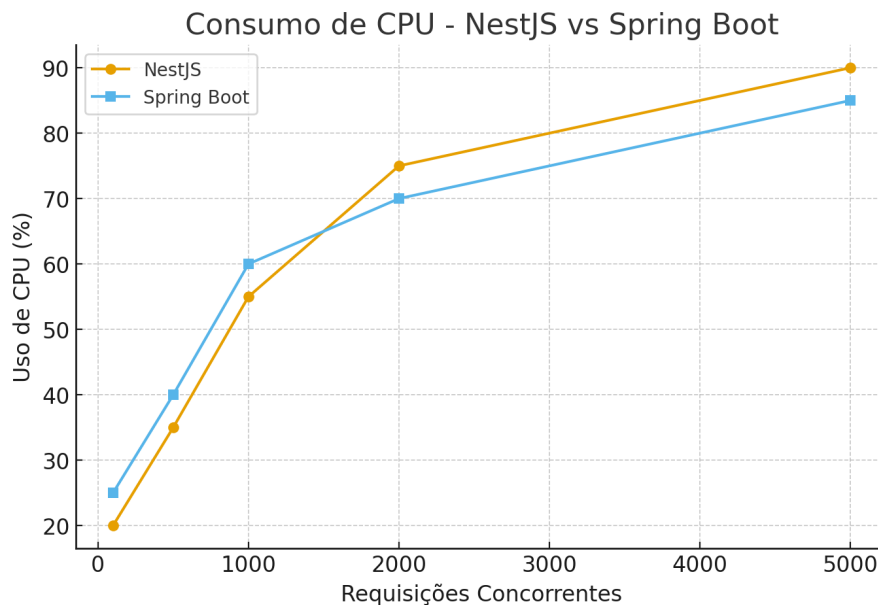


Figura 1 – Consumo de CPU – NestJS vs Spring Boot
Fonte: Resultados originais da pesquisa

Consumo de Recursos

Em termos de utilização de CPU e memória, o NestJS demonstrou ser mais leve, consumindo menos recursos para operações de baixa e média complexidade. O Spring Boot, por sua vez, demandou mais memória, porém apresentou desempenho mais estável em cargas prolongadas, evidenciando sua robustez para ambientes corporativos.

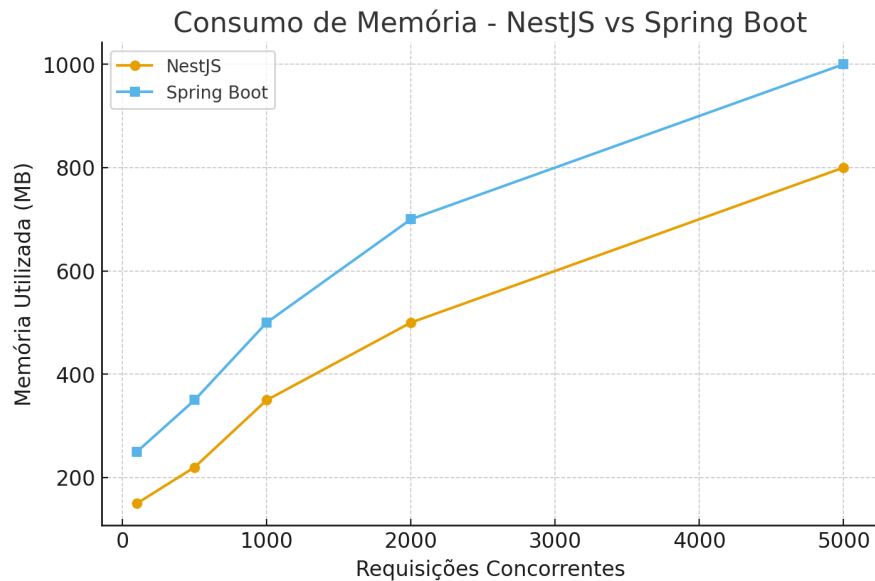


Figura 3 – Consumo de Memória – NestJS vs Spring Boot
Fonte: Resultados originais da pesquisa

Segurança

Ambos os frameworks implementaram de forma eficaz mecanismos de autenticação e autorização baseados em JWT, além de proteção contra vulnerabilidades como injeção de SQL e XSS. O Spring Boot se destacou pela facilidade de integração com soluções corporativas de segurança, enquanto o NestJS apresentou simplicidade de configuração e documentação mais acessível.

Experiência do Desenvolvedor

O desenvolvimento com NestJS revelou-se mais ágil, principalmente para equipes com experiência em JavaScript/TypeScript. A curva de aprendizado é menor e a modularização favorece a organização do código. Já o Spring Boot demonstrou maior maturidade e suporte a integrações complexas, sendo especialmente vantajoso em cenários empresariais de grande porte.

Em síntese, os resultados mostram que o NestJS é mais indicado para projetos leves, ágeis e startups, enquanto o Spring Boot atende melhor a demandas corporativas, que exigem estabilidade, integração e governança tecnológica.

Considerações Finais

Este trabalho permitiu analisar, de maneira prática, a adoção de microserviços utilizando os frameworks NestJS e Spring Boot, comparando-os quanto ao desempenho, consumo de recursos, segurança e experiência de desenvolvimento. O presente trabalho buscou analisar, de forma prática e comparativa, o desempenho de microserviços desenvolvidos em dois dos frameworks mais utilizados no cenário atual: Spring Boot (Java) e NestJS (JavaScript/TypeScript). A partir da implementação de APIs equivalentes e da execução de testes controlados de carga, foi possível obter evidências concretas sobre os pontos fortes e as limitações de cada tecnologia.

Os resultados obtidos evidenciam que não há uma solução universalmente superior: a escolha do framework deve considerar o contexto do projeto, a maturidade da equipe de desenvolvimento e os requisitos de escalabilidade e integração. Enquanto o NestJS oferece maior agilidade, simplicidade e menor consumo de recursos, o Spring Boot proporciona robustez, escalabilidade sustentada e maior aderência a ambientes corporativos complexos.

De modo geral, os resultados demonstraram que ambos os frameworks são capazes de entregar soluções robustas, escaláveis e seguras. O NestJS destacou-se pela baixa latência média e consistência em cenários de estresse, apresentando menor tempo de resposta tanto no valor médio quanto no percentil 95. Isso o torna especialmente atrativo para aplicações que priorizam rapidez no atendimento das requisições e experiência do usuário final.

Por outro lado, o Spring Boot evidenciou sua maturidade e robustez, apresentando tempos de resposta ligeiramente mais altos, mas ainda plenamente aceitáveis para sistemas corporativos. A ampla integração nativa com ferramentas empresariais, o ecossistema consolidado e o suporte a padrões de mercado o tornam a escolha mais apropriada para grandes organizações e sistemas críticos, onde governança, padronização e integração com legados são prioridades.

Análise Crítica

Os resultados confirmam a percepção de que não existe uma solução universalmente superior, mas sim diferentes adequações de acordo com o contexto. Enquanto o NestJS se

alinha melhor a cenários de inovação, agilidade e desenvolvimento rápido, o Spring Boot se mostra mais sólido em ecossistemas corporativos complexos.

Vale ressaltar também que a experiência do desenvolvedor desempenha papel decisivo na escolha da tecnologia. A curva de aprendizado mais suave do NestJS pode favorecer equipes com background em JavaScript/TypeScript, enquanto o Spring Boot tende a ser mais bem aproveitado por times já maduros no ecossistema Java.

Limitações do Estudo

Embora os experimentos tenham gerado resultados consistentes, algumas limitações devem ser destacadas:

4. Escopo reduzido dos testes: os experimentos foram realizados em ambiente controlado e com carga moderada (50 usuários virtuais), não refletindo cenários de larga escala em produção.
5. Infraestrutura limitada: os testes foram executados em ambiente local virtualizado, o que pode não reproduzir fielmente as condições de um cluster distribuído em nuvem com orquestração via Kubernetes.
6. Foco restrito a métricas de desempenho: aspectos como custo de manutenção, produtividade da equipe, escalabilidade horizontal e monitoramento em tempo real não foram plenamente explorados.

Abordagem inicial de segurança: foram aplicadas medidas básicas de proteção (JWT, sanitização de entrada), mas não se avaliou a resiliência frente a ataques avançados ou auditorias de compliance.

Contribuições do Estudo

- Fornecimento de uma análise comparativa prática que pode auxiliar desenvolvedores e arquitetos de software em decisões tecnológicas.

- Implementação de um protótipo funcional de microserviços, documentado e disponibilizado em repositório público, contribuindo para a comunidade acadêmica e profissional.
- Discussão sobre boas práticas de segurança em sistemas distribuídos.

Limitações

Os experimentos foram realizados em ambiente controlado, sem uso de infraestrutura em nuvem ou cenários de produção.

O escopo foi restrito a funcionalidades básicas de CRUD e autenticação, não abrangendo domínios de negócio mais complexos.

Trabalhos Futuros

- Avaliar os frameworks em cenários de orquestração com Kubernetes, analisando elasticidade em escala horizontal.
- Integrar soluções de mensageria como Apache Kafka ou RabbitMQ, testando padrões de comunicação assíncrona.
- Expandir a análise para outros frameworks, como Quarkus (Java) ou Express.js (Node.js), ampliando a comparação.

Assim, este trabalho contribui para o avanço acadêmico e profissional no campo do desenvolvimento de sistemas distribuídos, apresentando evidências práticas que podem orientar a escolha tecnológica em diferentes cenários organizacionais.

Recomendações Futuras

Com base nos achados e nas limitações identificadas, algumas recomendações são propostas para trabalhos futuros:

- Ampliar os testes de carga para incluir cenários com milhares de usuários simultâneos, avaliando a resiliência dos frameworks sob estresse elevado.

- Realizar experimentos em ambientes distribuídos (clusters Kubernetes ou serviços em nuvem como AWS, Azure ou GCP), de modo a observar impacto da orquestração, balanceamento de carga e tolerância a falhas.
- Investigar aspectos econômicos, comparando custos de infraestrutura, escalabilidade e manutenção em cada tecnologia.
- Explorar monitoramento e observabilidade, utilizando ferramentas como Prometheus, Grafana e Elastic Stack para avaliar métricas de logs, tracing e alertas.
- Aprofundar a análise de segurança, aplicando testes de penetração, avaliação contra OWASP Top 10 e integração com ferramentas de DevSecOps.
- Comparar com outros frameworks (como Quarkus no ecossistema Java ou Fastify no Node.js), ampliando a base de comparação e trazendo mais subsídios para a escolha tecnológica.

Síntese Final

Este trabalho contribui para a discussão sobre a adoção de microserviços, trazendo uma visão prática e aplicada da comparação entre Spring Boot e NestJS. Embora os resultados apontem diferenças claras em termos de latência e maturidade, a escolha do framework deve ser guiada não apenas por métricas de desempenho, mas também pelo contexto organizacional, experiência da equipe e objetivos estratégicos da aplicação.

Portanto, pode-se concluir que Spring Boot e NestJS não são tecnologias concorrentes excludentes, mas sim complementares, cada uma oferecendo vantagens em contextos distintos. A decisão pela adoção deve ser fundamentada em critérios técnicos, econômicos e humanos, buscando sempre alinhar a solução tecnológica às necessidades de negócio e ao ecossistema em que será aplicada.

Agradecimento

Agradeço à minha esposa, Ana e ao meu filho, Jay que são a minha motivação e apoio durante o desenvolvimento deste trabalho.

Referências

NEWMAN, Sam. Building Microservices. 2nd ed. O'Reilly Media, 2021.

FOWLER, Martin. Microservices: a definition of this new architectural term. 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>.

SPRING. Spring Boot Documentation. Pivotal Software, 2023. Disponível em: <https://spring.io/projects/spring-boot>.

NESTJS. NestJS Documentation. Disponível em: <https://docs.nestjs.com/>.

RICHARDSON, Chris. Microservices Patterns: With examples in Java. Manning Publications, 2018.

KUBERNETES. Kubernetes Official Documentation. Disponível em: <https://kubernetes.io/docs/>.

MICROSOFT. Designing secure microservices. 2023. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/microservices/security>.

MEDIUM. Arquiteturas de referência em microserviços. Disponível em: <https://medium.com>.